

64. Une grille

Deux index, et un calcul qu'on n'écrit pas.

1. Ce qu'il faut comprendre

`carte[1][c]` est rangé à plat, ligne après ligne, et le compilateur calcule `1 × LARGEUR + c`. C'est le calcul que tout le monde écrit à la main — et que tout le monde finit par écrire de travers le jour où la largeur change.

Le compilateur **exige les deux index** : `carte[3]` désignerait une rangée entière, et non un octet. Il le dit plutôt que de deviner.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **342 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage – la mise en route de la console
main()         170 o ← démarrage – ta fonction
VBlank          8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage   28 o ← image() – attend l'image suivante
AttendreVBlank  16 o ← démarrage – attend que l'écran soit libre
AdresseCase     10 o ← poser() – trouve l'adresse d'une case de l'écran
CopierTuiles    31 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte    14 o ← démarrage – vide les deux cartes de l'écran
Tuiles          16 o ← CopierTuiles – 1 lettre sur 43
```

Lettres gravées : A

Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, EffacerCases, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
poser() × 1      17 o sur place, dans main() + AdresseCase (10 o)
image() × 1      3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : range une petite salle (des murs tout autour) dans une GRILLE,
// puis la dessine avec des A (la tuile 1).
//
// Une grille est un tableau à deux dimensions :
//  uint8_t carte[8][10] = 8 lignes de 10 cases.
//  carte[1][c] = la case de la ligne 1, colonne c. 1 = un mur, 0 = rien.

#include <poser>    // pose une tuile sur une case du fond
```

2. UNE CONSTANTE — LARGEUR

```
const uint8_t LARGEUR = 10;           // 10 colonnes
```

3. UNE CONSTANTE — HAUTEUR

```
const uint8_t HAUTEUR = 8;           // 8 lignes
```

4. UN TABLEAU — CARTE

```
uint8_t carte[HAUTEUR][LARGEUR];    // la grille : 8 × 10 = 80 cases
```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

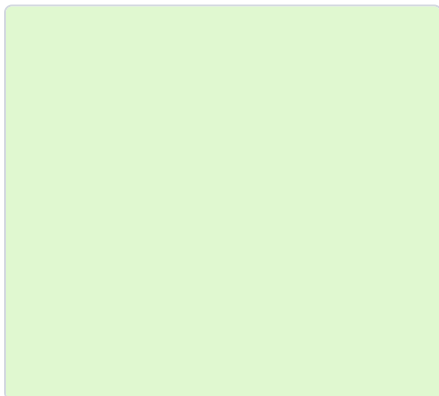
```
int main() {                          // Le jeu commence ici.
    // 1. Remplir la grille.
    for (uint8_t l = 0; l < HAUTEUR; l++) // chaque ligne l...
        for (uint8_t c = 0; c < LARGEUR; c++) // ... chaque colonne c :
            carte[l][c] = (l == 0 || l == HAUTEUR - 1 || c == 0 || c == LARGEUR - 1) ? 1 : 0;
                                                    // sur un bord (première ou dernière ligne,
                                                    // première ou dernière colonne) → 1, sinon → 0.
                                                    // « condition ? 1 : 0 » = SI vrai 1, SINON 0.

    // 2. Dessiner la grille.
    for (uint8_t l = 0; l < HAUTEUR; l++)
        for (uint8_t c = 0; c < LARGEUR; c++)
            if (carte[l][c]) poser(c + 5, l + 5, 1); // un mur → une tuile 1 à l'écran,
                                                        // décalée de 5 cases vers la droite
                                                        // et vers le bas.

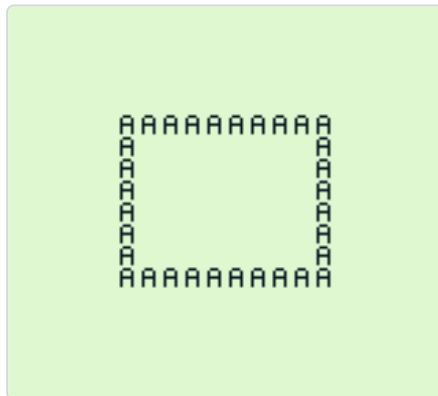
    while (true) {                          // Répète pour toujours :
        image();                             // attend l'image suivante.
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un rectangle creux, de dix sur huit, posé au milieu.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ les coins du rectangle sont posés

✓ les bords aussi

✓ le milieu est creux

✓ et rien ne dépasse

Ensuite : **65. Chercher dans un tableau** — chapitre 9 : Ranger ce qu'on manipule. · [gameboy3](#) — écrire une cartouche Game Boy en C++